

**Making Signals Naturally Difficult to
Classify with Supervised Learning
Algorithms via Model-Agnostic
Adversarial Perturbations**

UTD Team-1
Final Presentation



TEAM MEMBERS

Raviteja Avuthu	Raviteja.Avuthu@UTDallas.edu
Saipriya Nimmagadda	Saipriya.Nimmagadda@UTDallas.edu
Raghuveer Pachipulusu	Raghuveer.Pachipulusu@UTDallas.edu
Varun Potluri	Varun.Potluri@UTDallas.edu
Venkata Bhavana Vijay Srinivas Panchangam	Venkatabhavanavijaysrinivas.Panchangam@UTDallas.edu



Agenda

- Introduction
- Previous Work/Literature Review
- Plan
- Progress
- Results
- Challenges and issues
- Conclusion and Future Work
- Questions

INTRODUCTION



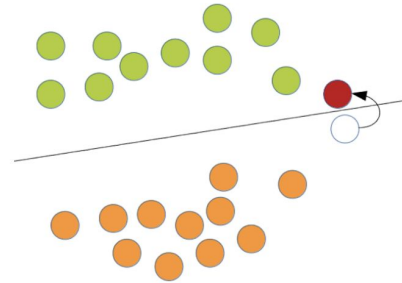
Data Evasion

Most common attack on machine learning model performed during inference.

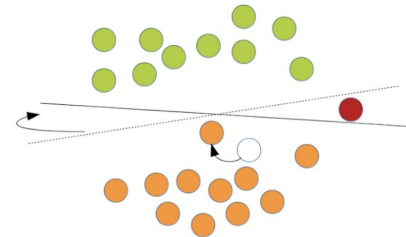


Data Poisoning

The attacker can exploit the ML model by poisoning the data before sending the data to the model for training.



Classical adversarial attack:
directly modifying the testing sample



Data poisoning:
modifying training samples intelligently

INTRO CONT.

Adversarial attacks involve generating slightly perturbed(noise) versions of the input data that fool the classifier(i.e, change its output)



x

“panda”

57.7% confidence

+ .007 ×



$\text{sign}(\nabla_x J(\theta, x, y))$

“nematode”

8.2% confidence

=

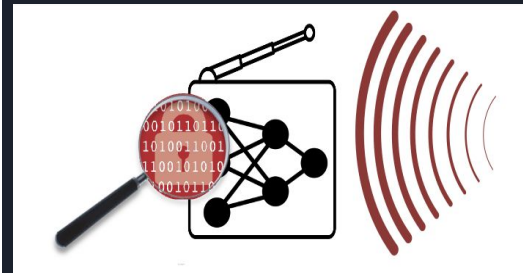
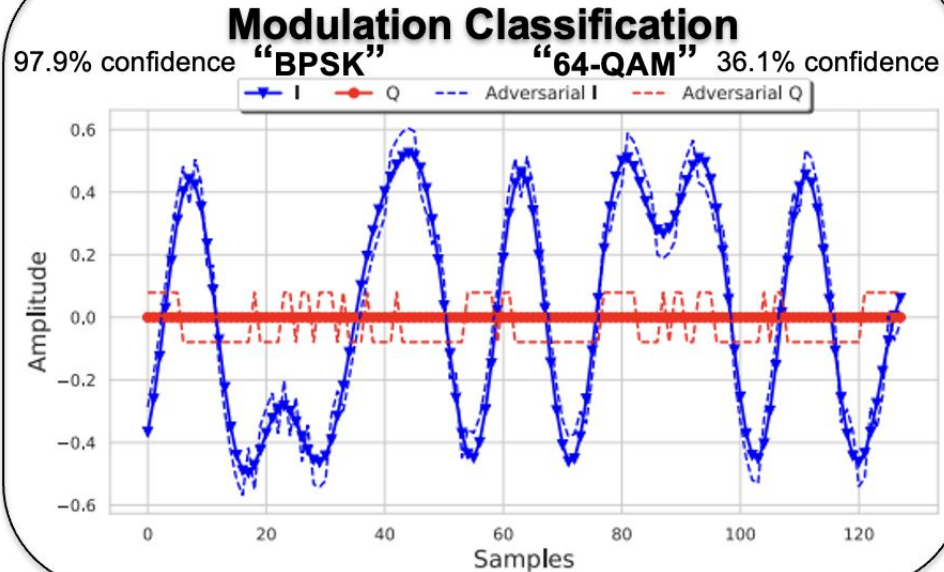


$x + \epsilon \text{sign}(\nabla_x J(\theta, x, y))$

“gibbon”

99.3 % confidence

Motivation





EXPLAINING ADVERSARIAL EXAMPLES

- $\tilde{x} = x + \eta$ where x is the input, $\tilde{x}$ is the adversarial input, η is the perturbation
- Expect same class prediction for x and $\tilde{x}$ if $\|\eta\| < \epsilon$, where ϵ is small enough to be discarded by the sensor or data storage apparatus associated with our problem.
- $w \cdot \tilde{x} = w \cdot x + w \cdot \eta$ w = weight vector, activation to grow by $w \cdot \eta$
- If w has n dimensions and the average magnitude of an element of the weight vector is m , then the activation will grow by mn
- Shows that a simple linear model can have adversarial examples if its input has sufficient dimensionality
- Neural networks linear behavior suggests that cheap, analytical perturbations of a linear model should also damage neural networks.



EXPLAINING ADVERSARIAL EXAMPLES CONT

- Adversarial example generated for one model is often misclassified by other models
- It is seen that multiple classifiers assign the same class to adversarial examples.
- Adversarial examples occur in contiguous regions of the 1-D subspace defined by the fast gradient sign method. Different models learn similar functions when trained to perform the same task.
- The direction of perturbation, rather than the specific point in space, matters most.
- Because it is the direction that matters most, adversarial perturbations generalize across different clean examples.



Plan

Phase 1: Get a basic understanding of Adversarial Perturbations and fundamentals

Phase 2: Know how to create a substitute model and how to craft adversarial samples. We want to identify the most powerful attack that can use (most incorrect classifications for a given power constraint) to attack an unknown, un-queryable classifier.

Phase 3: After identifying the attack, we now validate our attack on RML 2018 dataset forcing it to misclassify. Once we validate our attack model, then document this study for future work.



PREVIOUS WORK/LITERATURE REVIEW

References:

1. Flowers, Bryse, R. Michael Buehrer, and William C. Headley. "Evaluating adversarial evasion attacks in the context of wireless communications." IEEE Transactions on Information Forensics and Security 15 (2019): 1102-1113.
2. Practical black-box attacks against machine learning.
<https://dl.acm.org/doi/pdf/10.1145/3052973.3053009?ref=https://githubhelp.com>
3. Explaining and Harnessing Adversarial Examples.
<https://arxiv.org/pdf/1412.6572.pdf>
4. Query-efficient hard-label black-box attack: An optimization-based approach.



PROGRESS

Researched on different attack techniques:

- LBFGS

In target problem, L-BFGS Attack calculated approximate values of adversarial examples by line-searching with $c > 0$

We repeatedly solve this optimization problem for multiple values of c , adaptively updating c

- FGSM

FGSM attack is a *white-box* attack with the goal of *misclassification*

The fast gradient sign method uses the gradient of the loss function to determine in which direction the pixel's intensity should be changed (whether it should be increased or decreased) to minimize the loss function; then, it shifts all pixels simultaneously

Difference Between them:

- FGSM is designed primarily to be fast instead of producing very close adversarial examples



Cont...

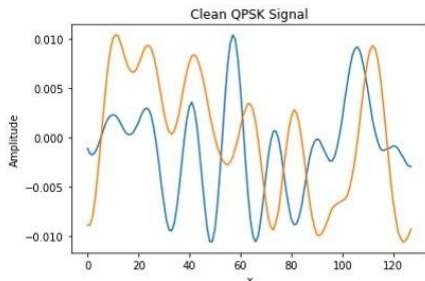
- Initially ran the model and tried to attack on 2016 dataset.
 - Was able to change the accuracy successfully
- Later tried to perform the same scenario on the 2018 dataset using tensorflow.
 - Initially faced few errors and the code was running long
 - Changed the scenarios to optimize the code and saved the trained model to make it load easily
- Faced few scenarios where we faced resource exhaustion when trying to calculate the accuracy after the attack
 - Tried different workflow to overcome the error

About DataSet

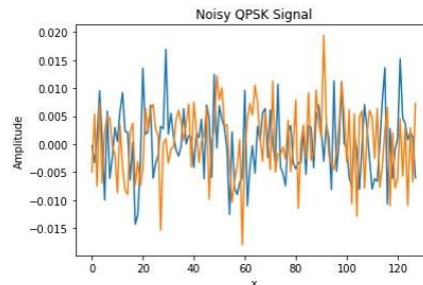
This dataset includes both synthetic simulated channel effects and over-the-air recordings of 24 digital and analog modulation types that have been heavily validated. Data is stored in hdf5 format as complex floating point values, with 2 million examples, each 1024 samples long.

DeepSig Dataset ,2018

```
classes = ['32PSK',  
'16APSK',  
'32QAM',  
'FM',  
'GMSK',  
'32APSK',  
'QPSK',  
'8ASK',  
'BPSK',  
'8PSK',  
'AM-SSB-SC',  
'4ASK',  
'16PSK',  
'64APSK',  
'128QAM',  
'128APSK',  
'AM-DSB-SC',  
'AM-SSB-WC',  
'64QAM',  
'QPSK',  
'256QAM',  
'AM-DSB-WC',  
'OOK',  
'16QAM']
```

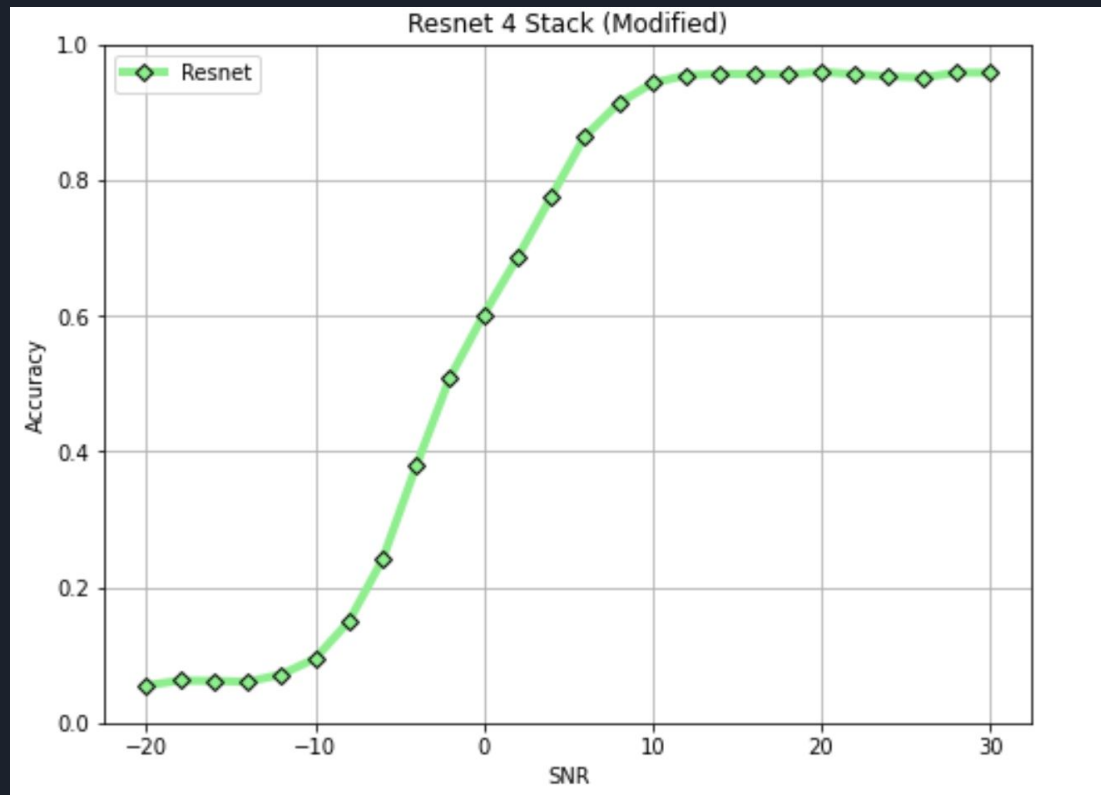


High VS Low
SNR

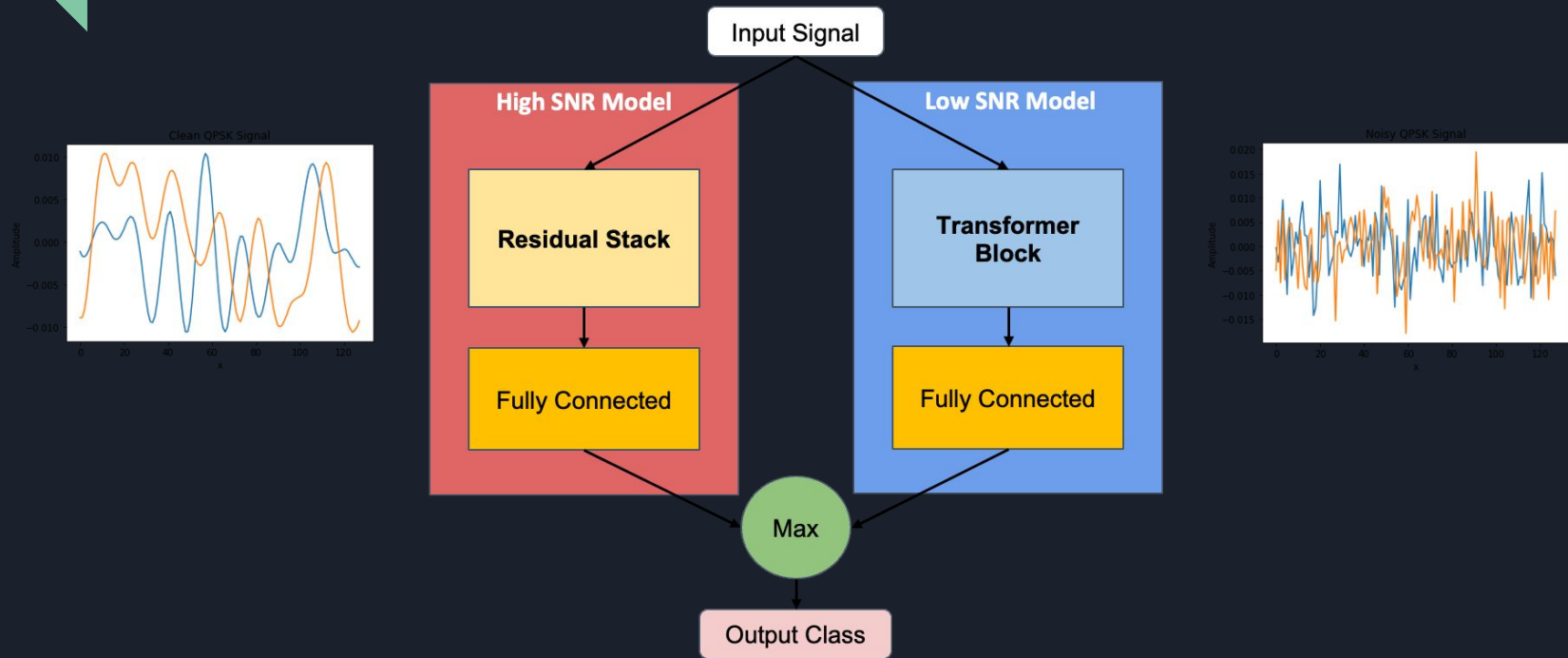


1. In the image below, we can see how drastically noise can affect our ability to recognize a signal.
2. We should be able to classify signals both in low SNR and high SNR.
3. We use the DeepSig Radio Signal Dataset, this dataset is pretty large (18GB). We will use 1.5 Million variation of signal modulation with respect to the SNR value.

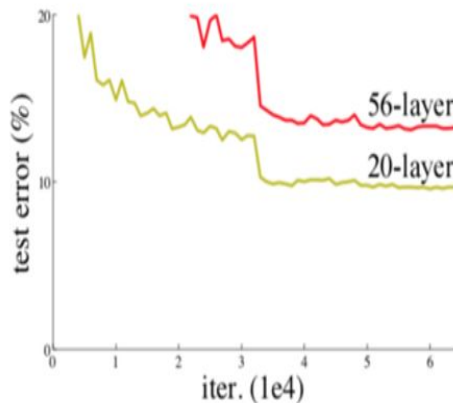
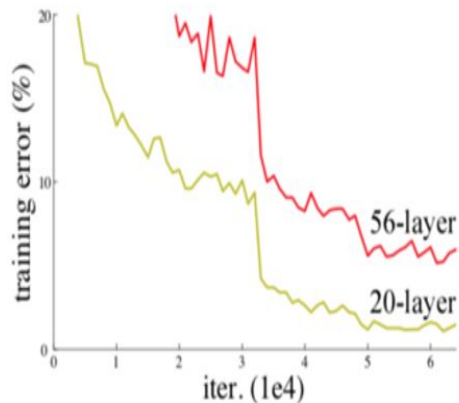
Baseline Model



PROPOSED METHOD



High SNR Model

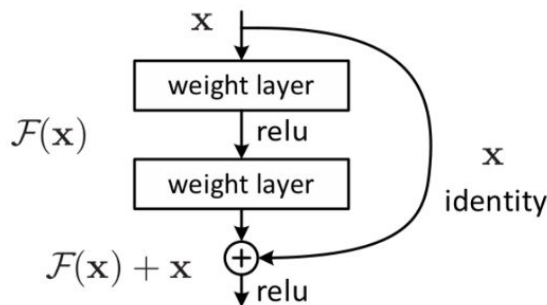


- One of the major benefits of a very deep network is that it can represent very complex functions.
- However, a huge barrier to training them is **vanishing gradients**: Gradient signal that goes to zero quickly, thus making gradient descent prohibitively slow

Two takeaways from residual block:

- Adding additional / new layers would not hurt the model's performance as regularisation will skip over them if those layers were not useful.
- If the additional / new layers were useful, even with the presence of regularisation, the weights or kernels of the layers will be non-zero and model performance could increase slightly.

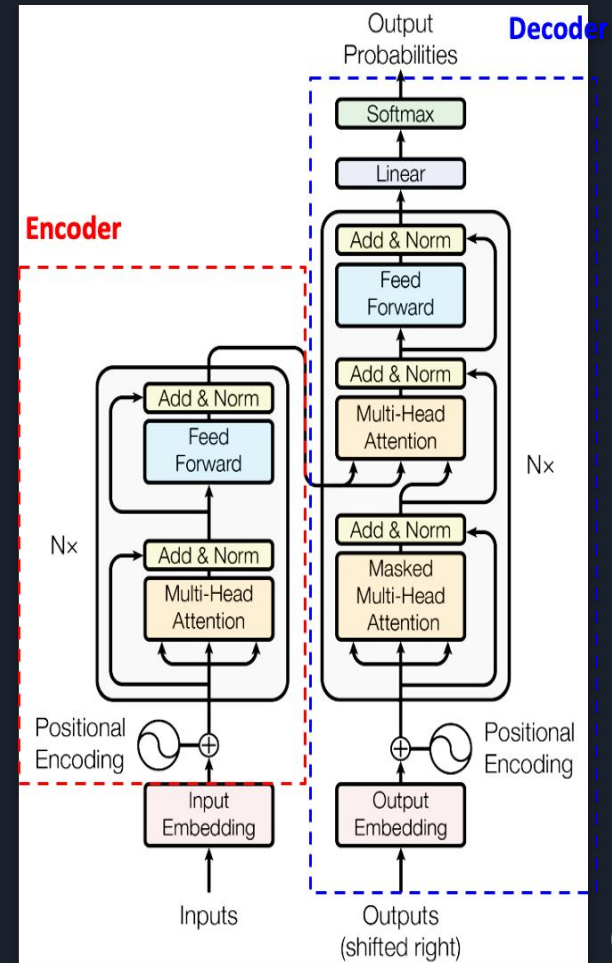
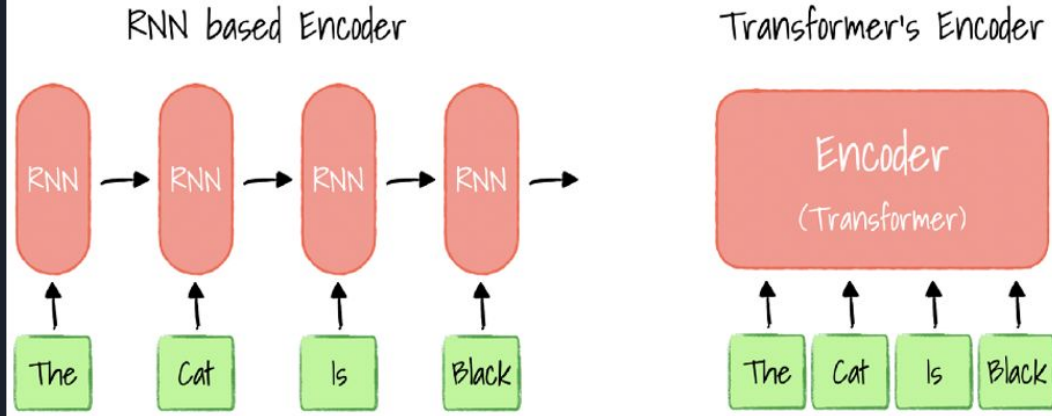
Solution :
Residual Block /
Identity Block



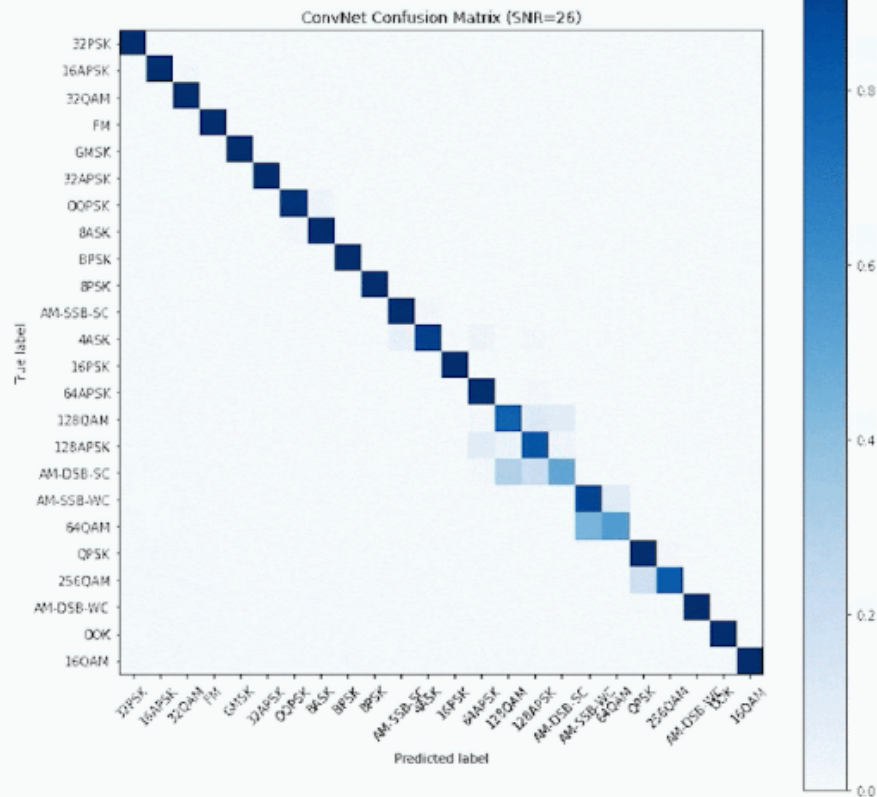
Low SNR Model

(Transformer, 2017)

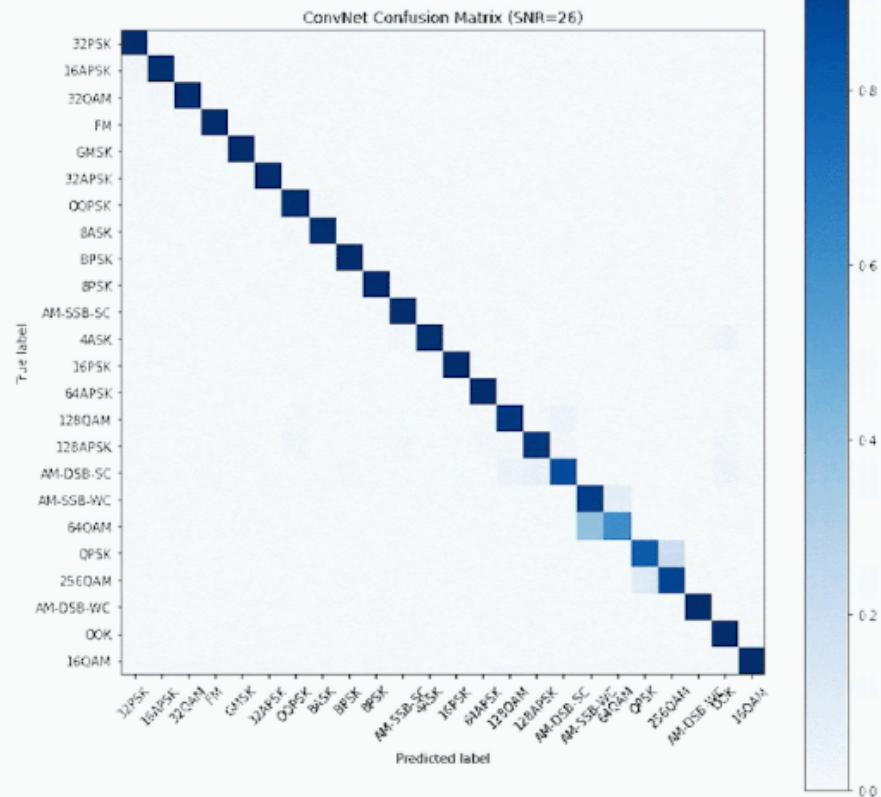
- Way faster training time and inference testing time (**parallelization**)
- Solved vanishing gradient problem (**encoder-decoder attention**)
- Context of data is well captured (**positional encoding & self attention mechanism**)



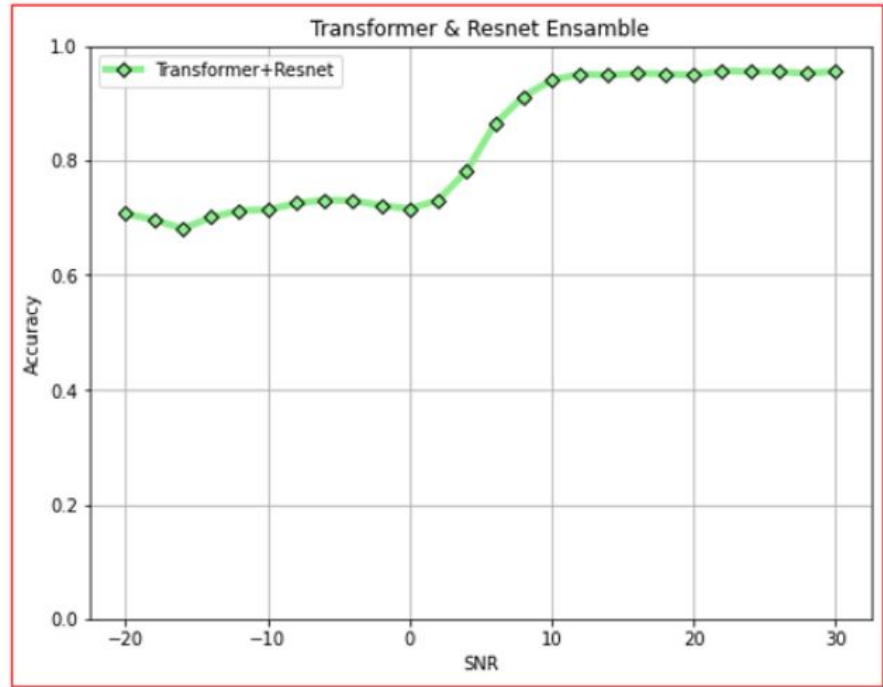
Baseline



Our Method



Our Model





Challenges faced

- We use the DeepSig Radio Signal Dataset, this dataset is pretty large (19GB). The dataset has 1.5 Million variation of signal modulation with respect to the SNR value. Due to the data set size as we have limited resources we tried implementing this on multiple systems.
- We have done extensive research to be able to classify signals both in low SNR and high SNR.
- Creating and training the model. As the dataset is huge, running it on our system with limited resource took almost 6 hrs for each Epoch. There were 5 Epochs'. We face difficulty here as the system would crash in between.
- To overcome this problem we saved the model right after it's execution and used the saved model in the next phase of project
- Running the attack on 20GB data. As the data is huge, we were not able to perform and test the attack on the given data set since our systems would not allow Tensorflow to use GPU resources.
- Resolution of errors
 - Huge barrier to train signal data is vanishing gradients
 - Kernel being dead at the last moment



Conclusion & Future Work

1. Was able to attack the RADIOML 2018.01A dataset but was tagged with the max pooling error and kernel being dead.
2. We were able to eliminate multiple false scenarios when we trained a model.
3. We have identified few particular scenarios and documented them so that they may not be repeated again to overcome the errors.
4. As a future work we might do an efficient attack on the model to change the accuracy and overcome the errors.



QUESTIONS

